Faculty of Engineering, Science & Technology

IQRA UNIVERSITY

Information Security

Project Report

[Fall-2025]

Name:	ID:
Muhammad Shees	IU05-0321-0285

Instructor Name: ***Dr.Muhammad Naveed***

Acknowledgment

”The best teacher teaches from the heart not from the BOOK. This report consist of a project.”

We would like to express our collective gratitude to our teacher for the invaluable guidance and mentorship provided throughout this project. Your support has empowered our group to develop this Packet Sniffing and analysis using Wireshark effectively and navigate its complexities with confidence. We are truly thankful for the time and effort you invested in teaching us, which allowed our team to successfully perform the work required for this report and develop technical skills that we can now apply together in real-world tasks.

Thank you.

Project Name: Packet Sniffing and Analysis using Wireshark

1. Project Overview

The goal of this project is to perform a deep-packet analysis of network traffic using **Wireshark**. The project focuses on understanding how different protocols interact within a local area network (LAN). By capturing real-time data, we aim to visualize the **TCP 3-Way Handshake**, analyze the **Protocol Hierarchy and I/O Graph**, and critically examine the security differences between unencrypted **HTTP** and encrypted protocols. This project serves as a practical demonstration of how network vulnerabilities can be exploited via packet sniffing and how encryption mitigates these risks.

---

2. Experimental Methodology

2.1 Tool Setup and Interface Selection

The initial phase involved launching Wireshark with administrative privileges to enable **Promiscuous Mode**.

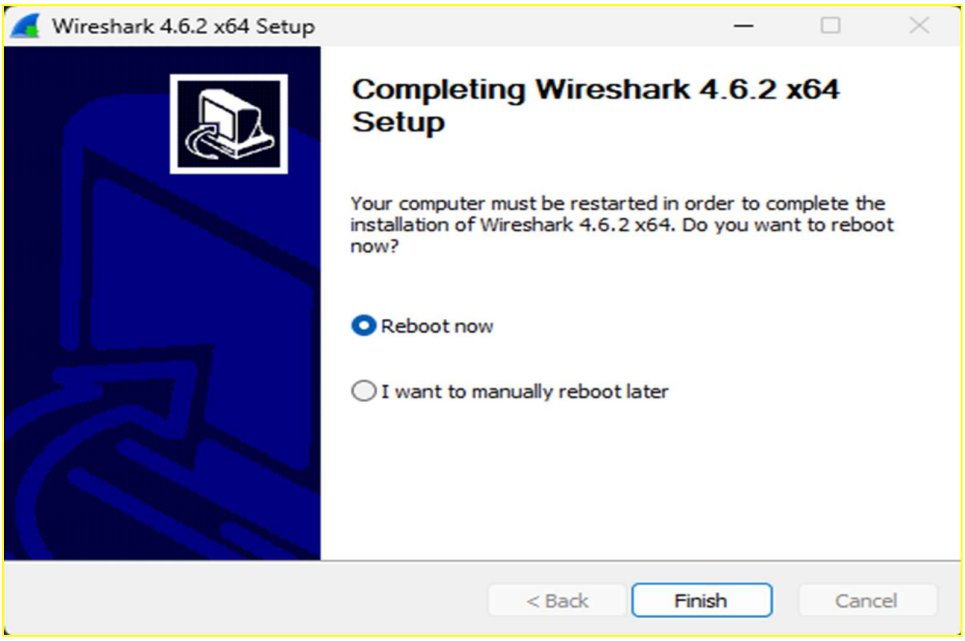


Figure1: Showing the welcome screen and the list of available interfaces.

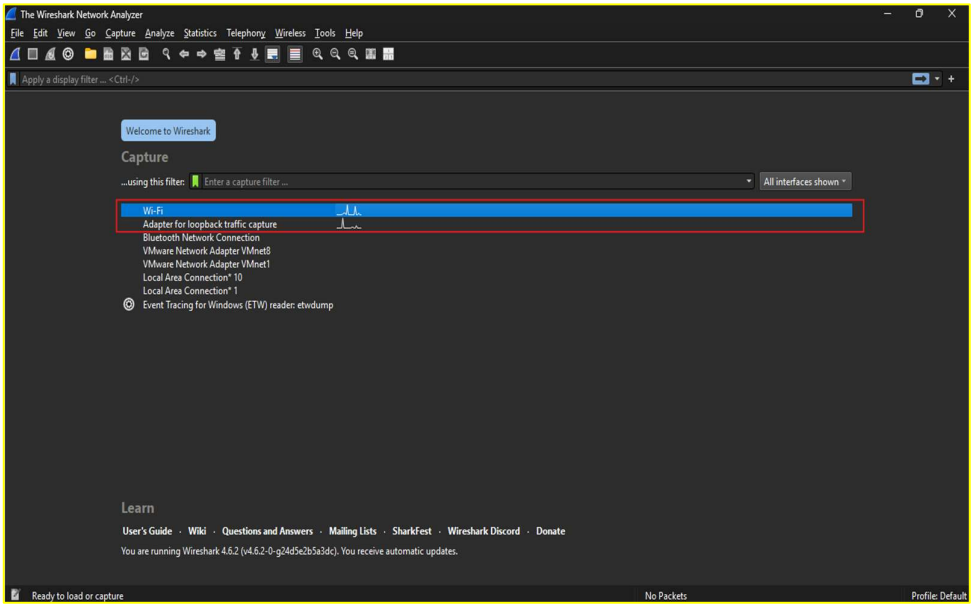


Figure2: Selecting the active wireless adapter to begin live capturing.

2.2 Live Data Capture

Once the interface was selected, the capture was initiated, recording all traffic flowing through the network adapter.

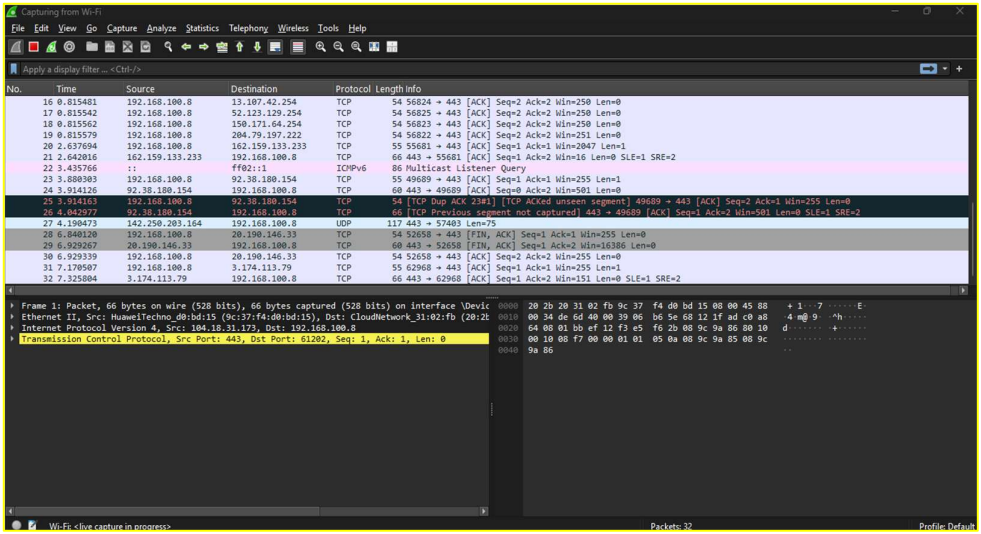


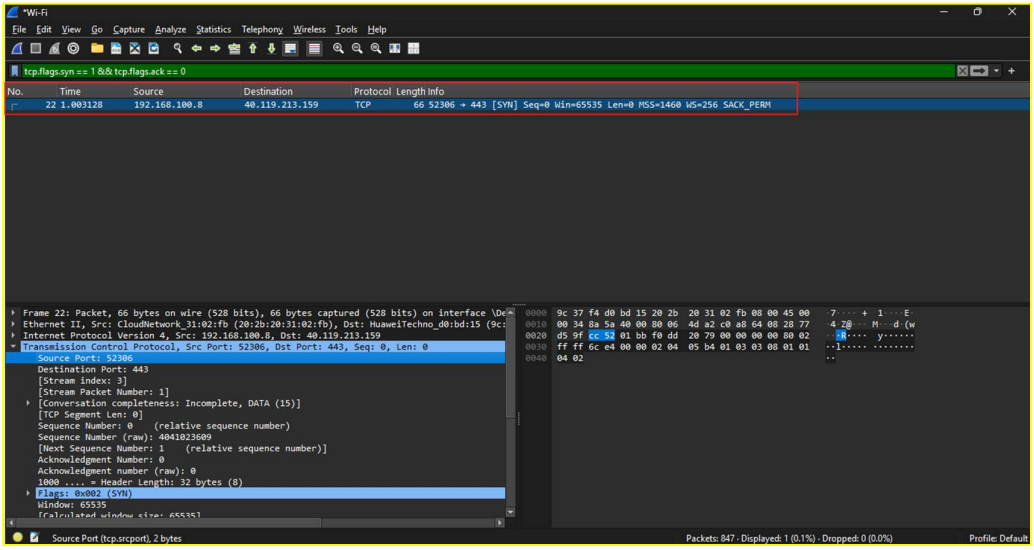
Figure3: The main packet list pane showing a mix of protocols (TCP, TLS, HTTP, etc.) in various colors.

3. TCP 3-Way Handshake Analysis

To visualize the establishment of a reliable connection, I first initiated a search on Google via my web browser. This action triggered a series of connection requests that were captured by Wireshark. I then isolated the initial handshake between my client and the server using specific flag filters.

Step1: SYN

- **Filter used:** tcp.flags.syn == 1 && tcp.flags.ack == 0
- **Description:** In this first step, the client sends a packet with the SYN flag set to request a new connection and synchronize sequence numbers.



Figur4: The client (my computer) initiates a connection request by sending a SYN packet to the server.

Step 2: SYN-ACK

- **Filter used:** tcp.flags.syn == 1 && tcp.flags.ack == 1
- **Description:** The server responds with a packet where both SYN and ACK flags are set, indicating it is ready to establish the connection.

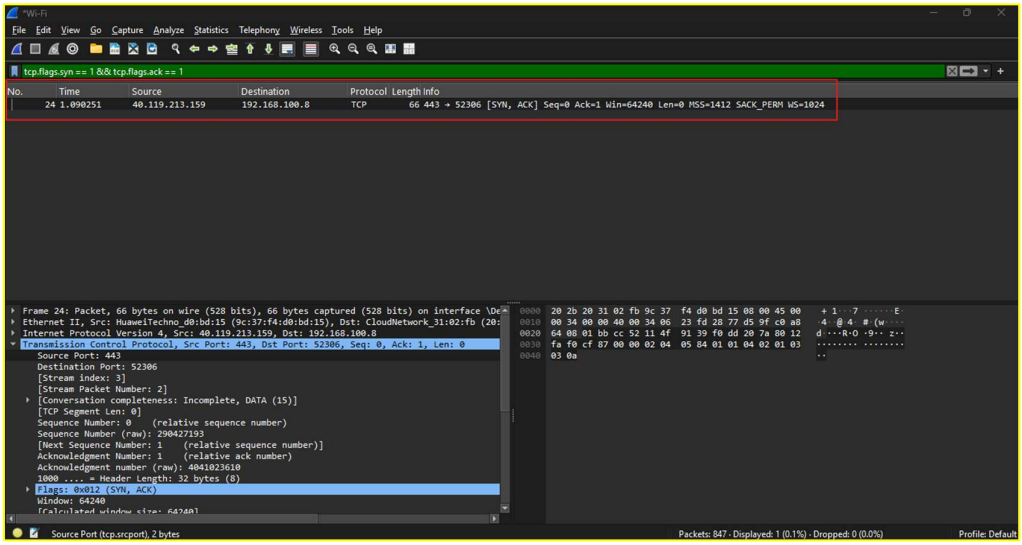


Figure5: The server acknowledges the request and sends its own synchronization signal.

Step 3: ACK

- **Filter used:** tcp.flags.ack == 1 && tcp.flags.syn == 0
- **Description:** The client sends a final ACK packet to the server. At this point, the connection is "Established," and data transfer can begin.

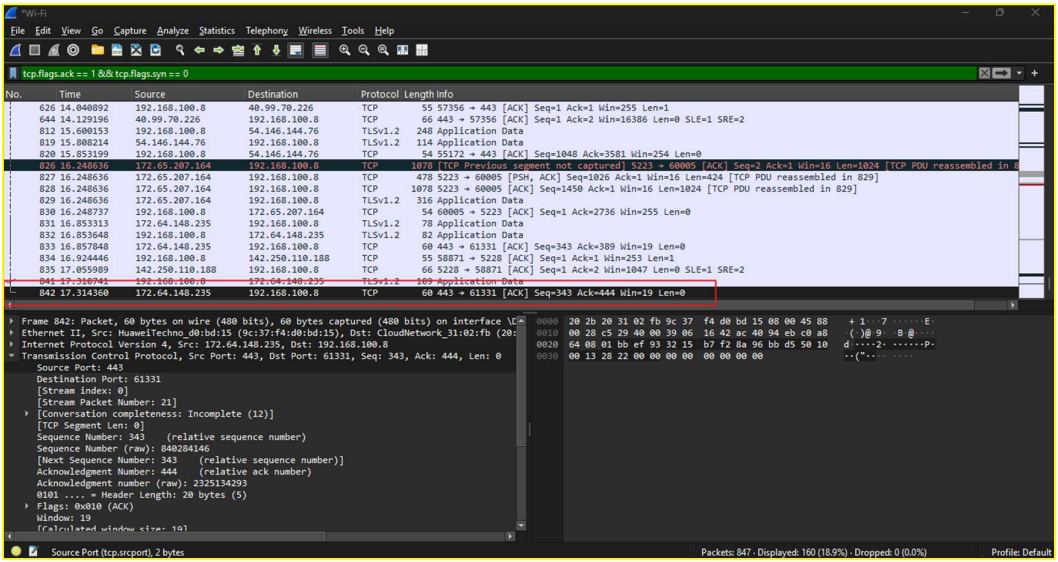


Figure6: The client confirms the connection, completing the 3-way handshake.

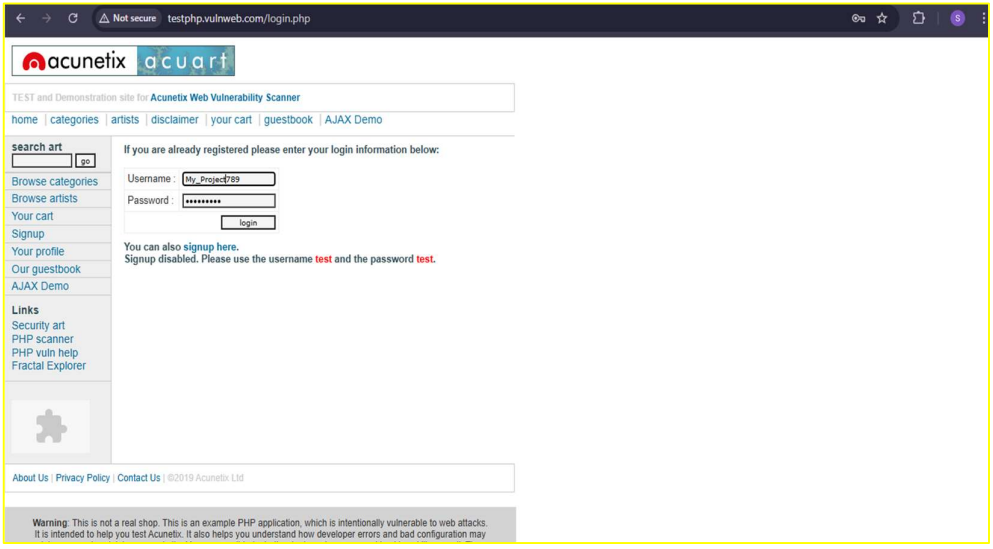
4. Targeted Protocol Analysis: HTTP Login

To simulate a real-world scenario, I accessed an unencrypted website and performed a login using the credentials:

- **Username:** My_Project789
- **Password:** secret123

4.1 Filtering for POST Requests

Since login data is typically sent to the server using the **HTTP POST** method, I applied the specific display filter: http.request.method == "POST".



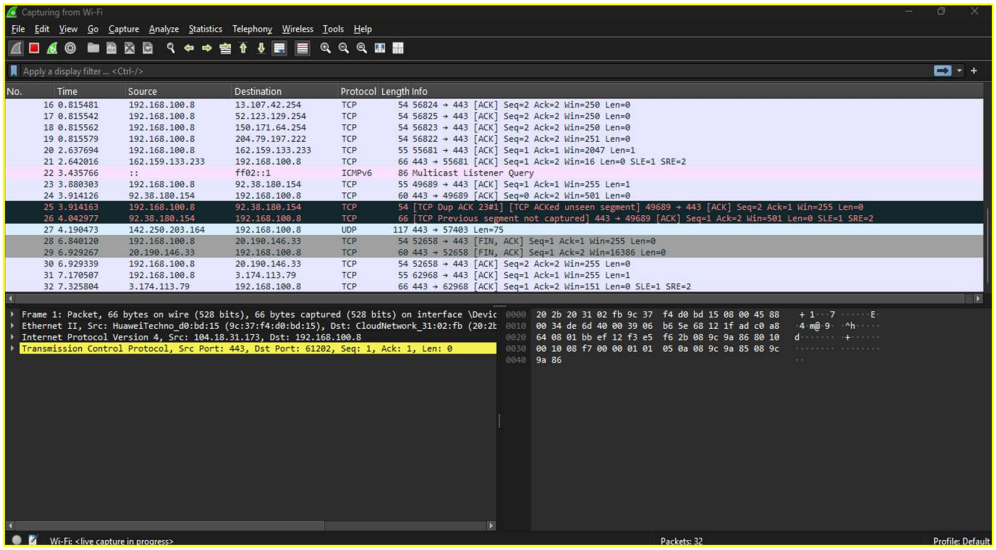


Figure7 and Figure8: The filtered view showing only the login submission packet.

4.2 Security Vulnerability Analysis (Plain Text Extraction)

By examining the **Packet Details Pane** of the POST request, I expanded the **"HTML Form URL Encoded"** section.

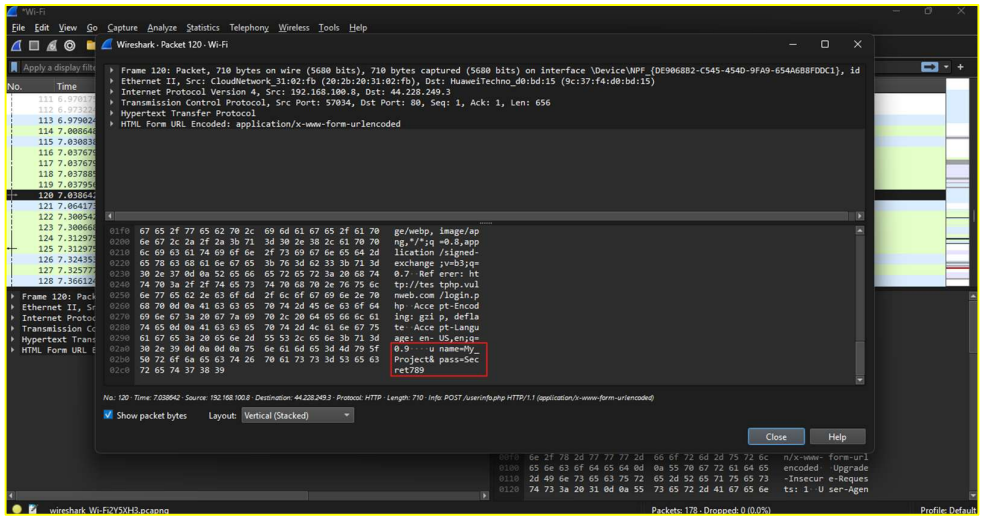


Figure9.

- **Analysis:** As observed in the screenshot, the credentials My_Project789 are clearly visible. This demonstrates that **HTTP** does not provide confidentiality. Any attacker on the same network using a sniffer could capture this sensitive information instantly.

5. Network Statistics and Visualization

5.1 Protocol Hierarchy

The **Protocol Hierarchy** provides a "family tree" of the captured data, showing which protocols consumed the most traffic.

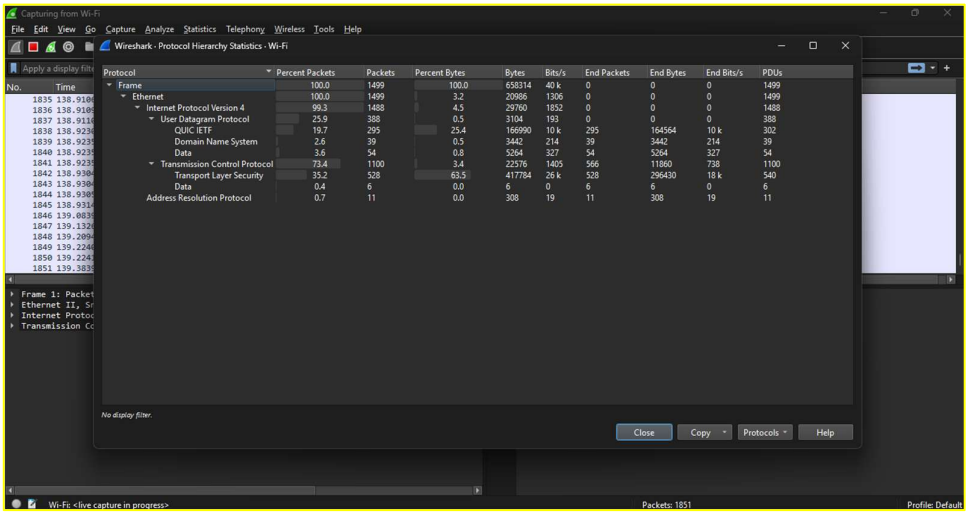


Figure10: Protocol Hierarchy

- **Interpretation:** This view shows the percentage of packets for each layer. In my capture, **TCP** dominated the traffic, while **UDP** (used for DNS) accounted for a smaller percentage.

5.2 I/O Graph Analysis

The **I/O Graph** visualizes the traffic density over the timeline of the capture.

Figure11: All Packets Graph.

Figure12: TCP Graph.

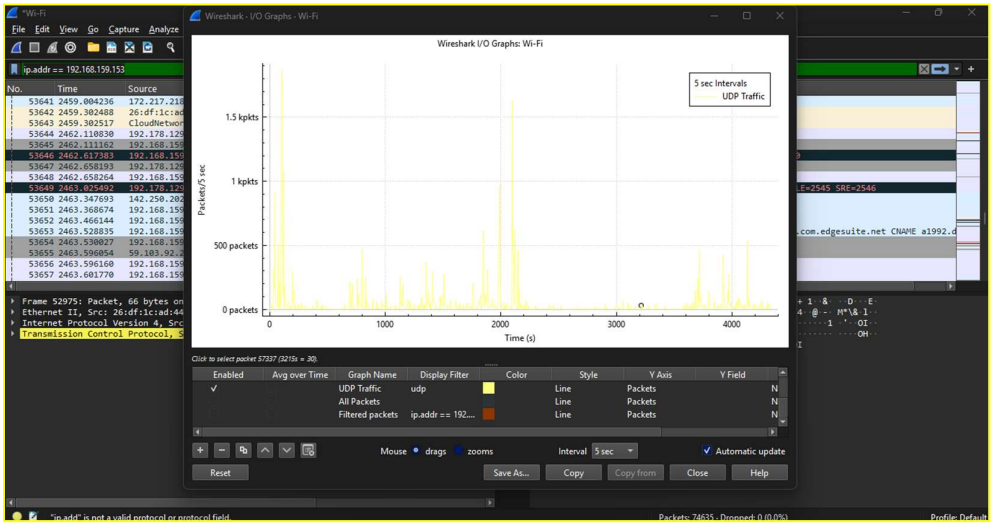


Figure13: UDP Graph.

Analysis:

- **All Packets (Black line):** Shows total network activity.
- **TCP (Red line):** Spiked during the web login and page loading, showing heavy data transfer.
- **UDP (Yellow line):** Remained low, representing the quick "question-and-answer" nature of DNS queries.

5.3 Specific Host Analysis

I filtered the capture to focus solely on my target machine's activity using the filter ip.addr == 192.168.159.153.

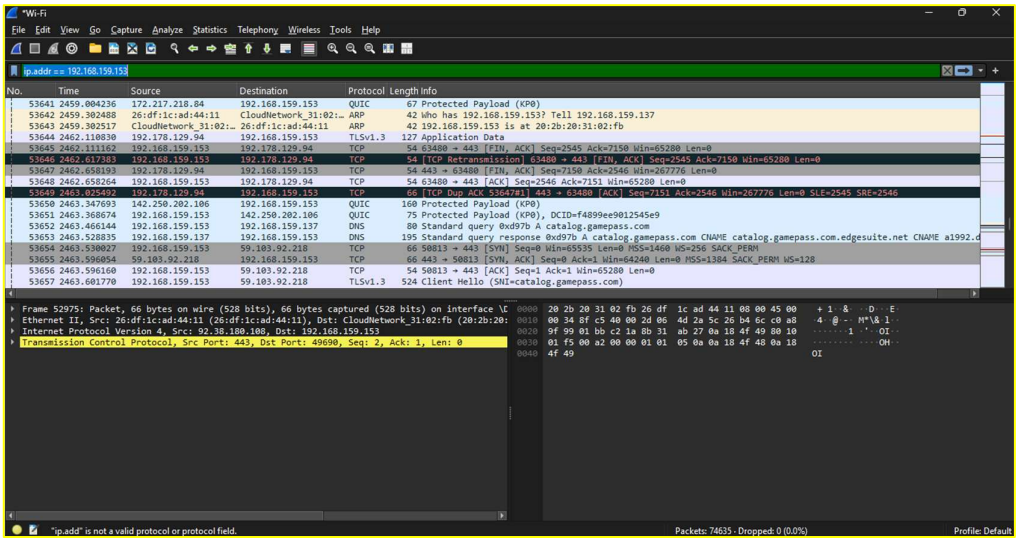


Figure14: Isolating all incoming and outgoing traffic for the specific host.

---

6. Conclusion

This project successfully demonstrates the mechanics of network communication. By visualizing the **TCP 3-way handshake** and protocol distribution via **I/O Graphs**, we gained a deep understanding of network behavior. Most importantly, the successful capture of a plain-text password from an HTTP session highlights the critical need for **TLS/HTTPS** encryption in modern web applications to ensure user privacy and data security.